程式設計
Variable, Expression, and Assignment Statements

張賢宗

人工智慧學系

本投影片修改自書商、林仲志教授之原始版本

Learning Objectives

- Variables, Expressions, and Assignment Statements
- Boolean Expressions
  - Building, Evaluating & Precedence Rules

widelab
Web Information and Data Engineering Laboratory

C++ Variables

int score;

0x00	
0x04	
0x08	
0x0C	
0x10	
0x14	

- C++ Identifiers
  - Keywords/reserved words vs. Identifiers
  - Case-sensitivity and validity of identifiers
  - Meaningful names!
- Variables
  - A memory location to store data for a program
  - Must declare all data before use in program

保留字/關鍵字

1. C語言的保留字(32個)：

auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while

2. C++額外的保留字(包括C++11及以後版本,約63個)：

asm, bool, catch, class, const_cast, delete, dynamic_cast, explicit, export, false, friend, inline, mutable, namespace, new, operator, private, protected, public, reinterpret_cast, static_cast, template, this, throw, true, try, typeid, typename, using, virtual, wchar_t

3. C++11及之後版本引入的新關鍵字： alignas, alignof, char16_t, char32_t, constexpr, decltype, noexcept, nullptr, static_assert, thread_local

4. C++20引入的新關鍵字： concept, requires, consteval, co_await, co_return, co_yield

Data Type

- 整數類型：
  - char：字符型，通常佔1字節
  - short：短整型
  - int：整型
  - long：長整型
  - long long：更長的整型（C99/C++11引入）
- 浮點類型：
  - float：單精度浮點型
  - double：雙精度浮點型
  - long double：擴展精度浮點型
- 布爾類型：
  - bool：布爾型（僅在C++中是內建類型，C99後在C中通過 <stdbool.h> 支持）
- 空類型：
  - void：表示無類型或空類型
- 寬字符類型： C++98
  - wchar_t：寬字符類型（在C++中是內建類型，在C中需要包含 <stddef.h>）
- C++11新增的字符類型：
  - char16_t：16位Unicode字符
  - char32_t：32位Unicode字符

Simple Types

資料類型	大小（位元組）	值的範圍
char	1	-128 到 127 或 0 到 255
unsigned char	1	0 到 255
short	2	-32,768 到 32,767
unsigned short	2	0 到 65,535
int	4	-2,147,483,648 到 2,147,483,647
unsigned int	4	0 到 4,294,967,295
long	4 或 8	-2,147,483,648 到 2,147,483,647 (32位系統) -9,223,372,036,854,775,808 到 9,223,372,036,854,775,807 (64位系統)
unsigned long	4 或 8	0 到 4,294,967,295 (32位系統) 0 到 18,446,744,073,709,551,615 (64位系統)
long long	8	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807
unsigned long long	8	0 到 18,446,744,073,709,551,615
float	4	1.2E-38 到 3.4E+38 (精度約為6-7位十進制數字)
double	8	2.3E-308 到 1.7E+308 (精度約為15-16位十進制數字)
long double	8, 12, 或 16	3.4E-4932 到 1.1E+4932 (精度約為18-19位十進制數字)
bool	1	true 或 false

Data Types

wchar_t	2 或 4	1個寬字符
char16_t	2	0 到 65,535
char32_t	4	0 到 4,294,967,295

int and long

- 在 C++ 中，long 和 int 是兩種基本的整數數據類型。它們之間的主要差異在於他們所能表達的數值範圍。不過，標準並未精確規定 int 和 long 的大小，而只是規定 long 不得小於 int。也就是說，long 的數值範圍應該大於或等於 int 的範圍。
- 在一個典型的 32 位元系統中：
  - int 4 個bytes，範圍從 -2,147,483,648 到 2,147,483,647。
  - long 4 個bytes ，範圍同 int。
- 而在一個典型的 64 位元系統中：
  - int 4 個bytes ，範圍從 -2,147,483,648 到 2,147,483,647。
  - long 8 個bytes ，範圍從 -9,223,372,036,854,775,808 到 9,223,372,036,854,775,807。
- 怎麼知道？

widelab
Web Information and Data Engineering Laboratory

Assigning Data

- Initializing data in declaration statement
  - Results "undefined" if you don't!
    - int myValue = 0;
- Assigning data during execution
  - Lvalues (left-side) & Rvalues (right-side)
    - Lvalues must be variables
    - Rvalues can be any expression
    - Example:
      distance = rate * time;
      Lvalue: "distance"
      Rvalue: "rate * time"

int score;

0x00	3951
0x04	4351
0x08	5313
0x0C	34
0x10	3135
0x14	2352

Assigning Data: Shorthand Notations

- Display, page 14

EXAMPLE	EQUIVALENT TO
`count += 2;`	`count = count + 2;`
`total -= discount;`	`total = total - discount;`
`bonus *= 2;`	`bonus = bonus * 2;`
`time /= rushFactor;`	`time = time/rushFactor;`
`change %= 100;`	`change = change % 100;`
`amount *= cnt1 + cnt2;`	`amount = amount * (cnt1 + cnt2);`

Data Assignment Rules

- Compatibility of Data Assignments
  - Type mismatches
    - General Rule: Cannot place value of one type into variable of another type
  - intVar = 2.99; // 2 is assigned to intVar!
    - Only integer part "fits", so that's all that goes
    - Called "implicit" or "automatic type conversion"
  - Literals
    - 2, 5.75, "Z", "Hello World"
    - Considered "constants": can't change in program

Literal Data

- Literals
  - Examples:
    - 2 // Literal constant int
    - 5.75 // Literal constant double
    - ' Z' // Literal constant char
    - "Hello World" // Literal constant c-string
- Cannot change values during execution
- Called "literals" because you "literally typed " them in your program!

Escape Sequences

- "Extend" character set
- Backslash, \ preceding a character
  - Instructs compiler: a special "escape character" is coming
  - Following character treated as "escape sequence char"
  - Display 1.3 next slide

Display 1.3 Some Escape Sequences (1 of 2)

Display 1.3 Some Escape Sequences

SEQUENCE	MEANING
\n	New line
\r	Carriage return (Positions the cursor at the start of the current line. You are not likely to use this very much.)
\t	(Horizontal) Tab (Advances the cursor to the next tab stop.)
\a	Alert (Sounds the alert noise, typically a bell.)
\\	Backslash (Allows you to place a backslash in a quoted expression.)

Display 1.3
Some Escape Sequences (2 of 2)

\'	Single quote (Mostly used to place a single quote inside single quotes.)
\"	Double quote (Mostly used to place a double quote inside a quoted string.)
The following are not as commonly used, but we include them for completeness:	
\v	Vertical tab
\b	Backspace
\f	Form feed
\?	Question mark

Constants

- Naming your constants
  - Literal constants are "OK", but provide little meaning
    - e.g., seeing 24 in a program, tells nothing about what it represents
- Use named constants instead
  - Meaningful name to represent data
    const int NUMBER_OF_STUDENTS = 24;
    - Called a "declared constant" or "named constant"
    - Now use it’ s name wherever needed in program
    - Added benefit: changes to value result in one fix

Arithmetic Operators: Display 1.4 Named Constant (1 of 2)

Standard Arithmetic Operators

- Precedence rules – standard rules

Display 1.4 Named Constant

```
1  #include <iostream>
2  using namespace std;
3
4  int main( )
5  {
6      const double RATE = 6.9;
7      double deposit;

8      cout << "Enter the amount of your deposit $";
9      cin >> deposit;
```

Arithmetic Operators: Display 1.4 Named Constant (2 of 2)

```
10        double newBalance;
11        newBalance = deposit + deposit*(RATE/100);
12        cout << "In one year, that deposit will grow to\n"
13             << "$" << newBalance << " an amount worth waiting for.\n";

14        return 0;
15    }
```

SAMPLE DIALOGUE

Enter the amount of your deposit $**100**
In one year, that deposit will grow to
$106.9 an amount worth waiting for.

Arithmetic Precision

- Precision of Calculations
  - VERY important consideration!
    - Expressions in C++ might not evaluate as you’ d "expect"!
  - "Highest-order operand" determines type of arithmetic "precision" performed
  - Common pitfall!

Arithmetic Precision Examples

- Examples:
  - 17 / 5 evaluates to 3 in C++!
    - Both operands are integers
    - Integer division is performed!
  - 17.0 / 5 equals 3.4 in C++!
    - Highest-order operand is "double type"
    - Double "precision" division is performed!
  - int intVar1 =1, intVar2=2;
    intVar1 / intVar2;
    - Performs integer division!
    - Result: 0!

Individual Arithmetic Precision

- Calculations done "one-by-one"
  - 1 / 2 / 3.0 / 4 performs 3 separate divisions.
    - First→ 1 / 2 equals 0
    - Then→ 0 / 3.0 equals 0.0
    - Then→ 0.0 / 4 equals 0.0!
- So not necessarily enough to change just "one operand" in a large expression
  - Must keep in mind all individual calculations that will be performed during evaluation!

Type Casting

- Casting for Variables
  - Can add ".0" to literals to force precision arithmetic, but what about variables?
    - We can't use "myInt.0"!
  - static_cast<double>intVar
  - Explicitly "casts" or "converts" intVar to double type
    - Result of conversion is then used
    - Example expression:
      doubleVar = static_cast<double>intVar1 / intVar2;
      - Casting forces double-precision division to take place among two integer variables!

Kahoot Time

- https://play.kahoot.it/v2/?quizId=faa6fc49-8be6-4173-a778-e52933cc82c5

Type Casting

- Two types
  - Implicit—also called "Automatic"
    - Done FOR you, automatically
      17 / 5.5
      This expression causes an "implicit type cast" to take place, casting the 17 → 17.0
  - Explicit type conversion
    - Programmer specifies conversion with cast operator
      (double)17 / 5.5
      Same expression as above, using explicit cast
      (double)myInt / myDouble
      More typical use; cast operator on variable

Shorthand Operators

- Increment & Decrement Operators
  - Just short-hand notation
  - Increment operator, ++
    intVar++; is equivalent to
    intVar = intVar + 1;
  - Decrement operator, --
    intVar--; is equivalent to
    intVar = intVar – 1;

Shorthand Operators: Two Options

- Post-Increment
  intVar++
  - Uses current value of variable, THEN increments it
- Pre-Increment
  ++intVar
  - Increments variable first, THEN uses new value
- "Use" is defined as whatever "context " variable is currently in
- No difference if "alone" in statement: intVar++; and ++intVar; → identical result

Post-Increment in Action

- Post-Increment in Expressions:

```
int         n = 2,
            valueProduced;
valueProduced = 2 * (n++);
cout << valueProduced << endl;
cout << n << endl;
```

  - This code segment produces the output:
    4
    3
  - Since post-increment was used

Pre-Increment in Action

- Now using Pre-increment:

```
int         n = 2,
            valueProduced;
valueProduced = 2 * (++n);
cout << valueProduced << endl;
cout << n << endl;
```

  - This code segment produces the output:
    6
    3
  - Because pre-increment was used

Kahoot Time

- https://play.kahoot.it/v2/?quizId=53a3337d-8911-4ac6-a1b2-8da0807ab980

Boolean Expressions: Display 2.1 Comparison Operators

- Logical Operators
  - Logical AND (&&)
  - Logical OR (||)

Display 2.1 Comparison Operators

MATH SYMBOL	ENGLISH	C++ NOTATION	C++ SAMPLE	MATH EQUIVALENT
=	Equal to	==	x + 7 == 2*y	x + 7 = 2y
≠	Not equal to	!=	ans != 'n'	*ans* ≠ *'n'*
<	Less than	<	count < m + 3	*count* < m + 3
≤	Less than or equal to	<=	time <= limit	*time* ≤ *limit*
>	Greater than	>	time > limit	*time* > *limit*
≥	Greater than or equal to	>=	age >= 21	*age* ≥ 21

Evaluating Boolean Expressions

- Data type bool
  - Returns true or false
  - true, false are predefined library consts
- Truth tables
  - Display 2.2 next slide

Evaluating Boolean Expressions: Display 2.2 Truth Tables

Display 2.2 Truth Tables

AND

Exp_1	*Exp_2*	*Exp_1* && *Exp_2*
true	true	true
true	false	false
false	true	false
false	false	false

OR

Exp_1	*Exp_2*	*Exp_1* || *Exp_2*
true	true	true
true	false	true
false	true	true
false	false	false

NOT

Exp	!(*Exp*)
true	false
false	true

Display 2.3 Precedence of Operators (1 of 4)

Display 2.3 Precedence of Operators

::	Scope resolution operator
.	Dot operator
->	Member selection
[]	Array indexing
( )	Function call
++	Postfix increment operator (placed after the variable)
--	Postfix decrement operator (placed after the variable)
++	Prefix increment operator (placed before the variable)
--	Prefix decrement operator (placed before the variable)
!	Not
-	Unary minus
+	Unary plus
*	Dereference
&	Address of
new	Create (allocate memory)
delete	Destroy (deallocate)
delete[]	Destroy array (deallocate)
sizeof	Size of object
( )	Type cast

Highest precedence (done first)

Display 2.3
Precedence of Operators (2 of 4)

*	Multiply
/	Divide
%	Remainder (modulo)
+	Addition
–	Subtraction
<<	Insertion operator (console output)
>>	Extraction operator (console input)

Lower precedence (done later)

Display 2.3 Precedence of Operators (3 of 4)

Display 2.3 Precedence of Operators

All operators in part 2 are of lower precedence than those in part 1.

<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal
!=	Not equal
&&	And
||	Or

Display 2.3 Precedence of Operators (4 of 4)

=	Assignment
+=	Add and assign
−=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulo and assign
? :	Conditional operator
throw	Throw an exception
,	Comma operator

Lowest precedence (done last)

Precedence Examples

- Arithmetic before logical
  - x + 1 > 2 || x + 1 < -3 means:
    - (x + 1) > 2 || (x + 1) < -3
- Short-circuit evaluation
  - (x >= 0) && (y > 1)
  - Be careful with increment operators!
    - (x > 1) && (y++)
- Integers as boolean values
  - All non-zero values → true
  - Zero value → false

Self Test

- What is the boolean expression result? (if time=0, limit=10)
  - KAHOOT TIME
  - https://play.kahoot.it/v2/?quizId=99ce782d-2ced-46d7-9c46-abb2569c4d2b